CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler 0x önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Mirac Alademiroğlu
Öğrenci numarası (SID)	5001240027
E-posta	miracalademiroglu@ogr.iu.edu.tr
Seçilen görev (G1–G5)	G1
Seçilen ISA ve sürümü (ör. RV32IM)	RV32IM
Derleyici ve sürümü	RISC-V (32-bits) gcc 13.2.0
Simülatör / araçlar	RARS v1.6
Git deposu adresi	https://github.com/Milemir/CMPE3209

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.
Not: SID sabiti yönergedeki 32-bit sınır kuralı uyarınca (5001240027 > 2^31 - 1) son 9 hane (1240027) olarak alınmıştır; S parametresi ise tam okul numarasından (5+0+0+1+2+4+0+0+2+7 = 21) hesaplanmıştır.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001240027%10	7
D2	SID'in son iki hanesi	5001240027%100	27
S	SID'in hanelerinin toplamı	5+0+0+1+2+4+0+0+2+7	21

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
N	24 + (D2 mod 25)	24+(27 mod 25)	26
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı		128/32 /Direct/4/| –
Write policy	d çift → write-back, d tek → write-through		Write-through
k	(S mod 7) + 3	(21 mod 7) +3	3
Interrupt türü	S çift → Timer, S tek → I/O	21%2 =1	I/O
f	1.0 + 0.1 × d GHz (varsayım)	1.0+0.1x7	1.7 GHz (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	20 + 5 x (7 mod 4)	T1 = 1 cycle (varsayım); T2 = 35 cycle (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + (21 mod 7)	2 çekirdek (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	T = 90 + (5001240027 mod 20)	97

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	5	$\log_2(32)$
Line (r) veya set (d)	2	$\log_2(4)$
Tag (s − r veya s − d)	25	27- 2
Toplam	32	5+2+25

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☒ G1 Biyomedikal sinyal alarmı
- ☐ G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

> *Sinyal filtreleme kaydırmalı pencere tamsayı hareketli ortalamasıyla, alarm tespiti ise T eşiğine göre doğrusal taramayla gerçekleştirilmiştir. Hesaplama ve sınır kontrolleri mov_avg , eşik denetimi ise check_alarm fonksiyonuna ayrılmıştır. Simülatörde bellek doğrulaması için N=26 elemanlı arr dizisi ve çıktı değişkenleri global alanda tanımlanmıştır.*

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

```
1: /* Mirac Alademiroğlu, 5001240027, G1 */
2: #define N 26
3: #define R 80
4: #define B 50
5: const int SID = 1240027;
6: unsigned int alarm_count = 0;
7: int first_alarm_idx = -1;
8: int arr[N];
9:
10: int mov_avg(int p3, int p2, int p1, int cur, int idx) {
11:     if (idx == 0) return cur;
12:     if (idx == 1) return (p1 + cur) / 2;
13:     if (idx == 2) return (p2 + p1 + cur) / 3;
14:
15:     int sum = p3 + p2 + p1 + cur;
16:     return sum >> 2;
17: }
18:
19: void check_alarm(int i, int s, int T) {
20:     if (s > T) {
21:         alarm_count++;
22:         if (first_alarm_idx == -1) {
23:             first_alarm_idx = i;
24:         }
25:     }
26: }
27:
28: int main() {
29:     unsigned int x = SID;
30:     int i;
31:
32:     for (i = 0; i < N; i++) {
33:         x = (1664525 * x + 1013904223);
34:         arr[i] = (((x >> 16) & 0xFFFF) % R) + B;
35:     }
36:
37:     int p1 = 0, p2 = 0, p3 = 0;
38:     for (i = 0; i < N; i++) {
39:         int cur = arr[i];
40:         arr[i] = mov_avg(p3, p2, p1, cur, i);
41:         p3 = p2;
42:         p2 = p1;
43:         p1 = cur;
44:     }
45:
```

```
46:     int T = 90 + (SID % 20);
47:     for (i = 0; i < N; i++) {
48:         check_alarm(i, arr[i], T);
49:     }
50:
51:     return 0;
52: }
```

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	8, 38, 47	Global arr tanımlanmış (s. 8); 1. geçişte hareketli ortalama filtreleme (s. 38), 2. geçişte alarm taraması (s. 47) yapılmıştır.
Y2	Koşullu dallanma	20	if (s > T) satırında (s. 20) eşik kontrolü ve sınır durumlarında (s. 11–13) koşullu branch sağlanmıştır.
Y3	Parametreli, değer döndüren fonksiyon çağrısı	10, 40	Tanımlanan mov_avg fonksiyonu (s. 10) mov_avg(p3, p2, p1, cur, i) şeklinde (s. 40) çağrılmıştır.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	15, 16, 33, 34	mov_avg içinde + (s. 15), >> (s. 16); LCG içinde *, + (s. 33), >>, & (AND maskeleme) ve % (s. 34) kullanılmıştır.
Y5	Dizi elemanına store	40	arr[i] = mov_avg(...) ataması ile (s. 40) dizi elemanlarına bellek yazması (store) gerçekleştirilmiştir.
Y6	const int SID ve hesapta kullanımı	5, 29, 46	const int SID tanımlanmış (s. 5); LCG başlangıcında x = SID (s. 29) ve eşik hesabında T = 90 + (SID % 20) (s. 46) kullanılmıştır.
Y7	Sonucun tutulduğu global değişken(ler)	6, 7, 21, 23	Çıktı sonuçları global alarm_count (s. 21) ve first_alarm_idx (s. 23) değişkenlerinde saklanmıştır.

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
SID	0x10010000	0x0012ebdb (1240027)	output_screenshot
alarm_count	0x10010004	0x0000000b (11)	output_screenshot
first_alarm_idx	0x10010008	0x00000000 (0)	output_screenshot

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
arr[0]	0x1001000c	0x00000073 (115)	output_screenshot

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	19–26 (Çağrı: 48)
Başlangıç ve bitiş adresi (hex)	0x004000c4 – 0x00400134
Dinamik instruction sayısı	13
Hangi döngü iterasyonu	i = 3

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.s):

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
5	const int SID = 1240027;	SID: .word 1240027	0x10010000 – 0x10010003
6	unsigned int alarm_count = 0;	alarm_count: .space 4	0x10010004 – 0x10010007
7	int first_alarm_idx = -1;	first_alarm_idx: .word -1	0x10010008 – 0x1001000b
8	int arr[N];	arr: .space 104	0x1001000c – 0x10010073
10	int mov_avg(int p3, int p2, int p1, int cur, int idx)	addi sp, sp, -64 sw s0, 60(sp) addi s0, sp, 64 sw a0, -36(s0) sw a1, -40(s0) sw a2, -44(s0) sw a3, -48(s0) sw a4, -52(s0)	0x00400000 – 0x0040001c
11	if (idx == 0) return cur;	lw a5, -52(s0) bne a5, zero, .L2 lw a5, -48(s0)	0x00400020 – 0x0040002c

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		j .L3	
12	if (idx == 1) return (p1 + cur) / 2;	lw a4, -52(s0) li a5, 1 bne a4, a5, .L4 lw a4, -44(s0) lw a5, -48(s0) add a5, a4, a5 srli a4, a5, 31 add a5, a4, a5 srai a5, a5, 1 j .L3	0x00400030 – 0x00400054
13	if (idx == 2) return (p2 + p1 + cur) / 3;	lw a4, -52(s0) li a5, 2 bne a4, a5, .L5 lw a4, -40(s0) lw a5, -44(s0) add a4, a4, a5 lw a5, -48(s0) add a4, a4, a5 li a5, 3 div a5, a4, a5 j .L3	0x00400058 – 0x00400080
15	int sum = p3 + p2 + p1 + cur;	lw a4, -36(s0) lw a5, -40(s0) add a4, a4, a5 lw a5, -44(s0) add a5, a4, a5 lw a4, -48(s0) add a5, a4, a5 sw a5, -24(s0)	0x00400084 – 0x004000a0
16	return sum >> 2;	lw a5, -24(s0) srai a5, a5, 2 sw a5, -20(s0) lw a5, -20(s0) mv a0, a5 lw s0, 60(sp) addi sp, sp, 64 jr ra	0x004000a4 – 0x004000c0
19	void check_alarm(i	addi sp, sp, -32 sw s0, 28(sp)	0x004000c4 – 0x004000d8

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
	nt i, int s, int T)	addi s0, sp, 32 sw a0, -20(s0) sw a1, -24(s0) sw a2, -28(s0)	
20	if (s > T)	lw a4, -24(s0) lw a5, -28(s0) ble a4, a5, .L9	0x004000dc – 0x004000e4
21	alarm_count++;	la a5, alarm_count lw a5, 0(a5) addi a4, a5, 1 la a5, alarm_count sw a4, 0(a5)	0x004000e8 – 0x00400100
22	if (first_alarm_idx == -1)	la a5, first_alarm_idx lw a4, 0(a5) li a5, -1 bne a4, a5, .L9	0x00400104 – 0x00400114
23	first_alarm_idx = i;	la a5, first_alarm_idx lw a4, -20(s0) sw a4, 0(a5)	0x00400118 – 0x00400124
25	}	nop lw s0, 28(sp) addi sp, sp, 32 jr ra	0x00400128 – 0x00400134
28	int main()	addi sp, sp, -48 sw ra, 44(sp) sw s0, 40(sp) addi s0, sp, 48	0x00400138 – 0x00400144
29	unsigned int x = SID;	li a5, 1241088 addi a5, a5, -1061 sw a5, -20(s0)	0x00400148 – 0x00400154
30	int i;	—	—
32	for (i = 0; i < N; i++)	sw zero, -24(s0) j .L11 lw a5, -24(s0) addi a5, a5, 1 sw a5, -24(s0) lw a4, -24(s0)	0x00400158 – 0x0040015c 0x004001c4 – 0x004001d8

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		li a5, 25 ble a4, a5, .L12	
33	x = (1664525 * x + 1013904223);	lw a4, -20(s0) li a5, 1662976 addi a5, a5, 1549 mul a4, a4, a5 li a5, 1013903360 addi a5, a5, 863 add a5, a4, a5 sw a5, -20(s0)	0x00400160 – 0x00400184
34	arr[i] = (((x >> 16) & 0xFFFF) % R) + B;	lw a5, -20(s0) srli a4, a5, 16 li a5, 65535 and a4, a4, a5 li a5, 80 remu a5, a4, a5 addi a5, a5, 50 mv a3, a5 la a4, arr lw a5, -24(s0) slli a5, a5, 2 add a5, a4, a5 sw a3, 0(a5)	0x00400188 – 0x004001c0
37	int p1 = 0, p2 = 0, p3 = 0;	sw zero, -28(s0) sw zero, -32(s0) sw zero, -36(s0)	0x004001dc – 0x004001e4
38	for (i = 0; i < N; i++)	sw zero, -24(s0) j .L13 lw a5, -24(s0) addi a5, a5, 1 sw a5, -24(s0) lw a4, -24(s0) li a5, 25 ble a4, a5, .L14	0x004001e8 – 0x004001ec 0x0040025c – 0x00400270
39	int cur = arr[i];	la a4, arr lw a5, -24(s0) slli a5, a5, 2 add a5, a4, a5 lw a5, 0(a5)	0x004001f0 – 0x00400208

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		sw a5, -44(s0)	
40	arr[i] = mov_avg(p3, p2, p1, cur, i);	lw a4, -24(s0) lw a3, -44(s0) lw a2, -28(s0) lw a1, -32(s0) lw a0, -36(s0) call mov_avg mv a3, a0 la a4, arr lw a5, -24(s0) slli a5, a5, 2 add a5, a4, a5 sw a3, 0(a5)	0x0040020c – 0x00400240
41–43	p3 = p2; p2 = p1; p1 = cur;	lw a5, -32(s0) sw a5, -36(s0) lw a5, -28(s0) sw a5, -32(s0) lw a5, -44(s0) sw a5, -28(s0)	0x00400244 – 0x00400258
46	int T = 90 + (SID % 20);	li a5, 1241088 addi a4, a5, -1061 li a5, 20 rem a5, a4, a5 addi a5, a5, 90 sw a5, -40(s0)	0x00400274 – 0x0040028c
47	for (i = 0; i < N; i++)	sw zero, -24(s0) j .L15 lw a5, -24(s0) addi a5, a5, 1 sw a5, -24(s0) lw a4, -24(s0) li a5, 25 ble a4, a5, .L16	0x00400290 – 0x00400294 0x004002c4 – 0x004002d8
48	check_alarm(i, arr[i], T);	la a4, arr lw a5, -24(s0) slli a5, a5, 2 add a5, a4, a5 lw a5, 0(a5) lw a2, -40(s0) mv a1, a5	0x00400298 – 0x004002c0

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		lw a0, -24(s0) call check_alarm	
51	return 0;	li a5, 0 mv a0, a5 lw ra, 44(sp) lw s0, 40(sp) addi sp, sp, 48 li a7, 10 ecall	0x004002dc – 0x004002ec 0x004002f0 – 0x004002f4

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x00400000	760	mov_avg (196 B), check_alarm (116 B), main (448 B) fonksiyonlarının yürütülebilir makine kodları
.data / .bss	0x10010000	116	SID (4 B), alarm_count (4 B), first_alarm_idx (4 B), arr[26] (104 B) global değişken verileri

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

> *Von Neumann mimarisinde yürütülebilir program kodları (.text / 0x00400000) ile program verileri (.data / 0x10010000) ayrı fiziksel bellekler yerine aynı birleşik adres uzayında (unified address space) saklanır; işlemci (CPU) her iki alana da ortak sistem veriyolu (bus) ve standart adresleme mekanizmasıyla erişir.*
> *(Not 1: Normalde derleyiciler 'const int SID' gibi sabitleri salt-okunur .rodata segmentine yerleştirir; ancak asm_O0.s dosyasında simülatörde tüm değişken ve çıktıların tek bir data segment penceresinde (0x10010000) topluca izlenebilmesi amacıyla .data*

> *segmentinde konumlandırılmıştır.)*
> *(Not 2: RARS, 32 bitlik 'li' pseudo-instruction'ını her zaman koşulsuz olarak 'lui' + 'addi' olarak iki instruction'a açar. Yüklenen değerin alt 12 biti sıfır olduğunda (örneğin 1241088 = 0x0012F000 sabitinde), RARS alt kısım için 'addi x15, x15, 0' komutunu üretir (0x0040014c ve 0x00400168 adresleri). 'la' pseudo-instruction'ı ise 'auipc' + 'addi' olarak açılır. Sunulan döküm RARS Text Segment Window çıktısı olup çalışma anındaki gerçek bellek içeriğini yansıtmaktadır.)*
> *(Not 3: Satır 51'in sonundaki 'li a7,10; ecall' komutları C kaynak kodundan doğrudan üretilmemiş olup, RARS simülatör ortamında programın tanımsız bellek bölgelerine taşmadan temiz bir şekilde sonlanması (exit syscall) için eklenmiştir.)*

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	13	2	-O0 stack frame açıp değişkenleri stack belleği üzerinden işlerken, -O1 stack frame açmaz ve parametreleri doğrudan register üzerinden test eder.
Load sayısı	3	0	-O0 argümanları ve s0 frame pointer'ı stack belleğinden okurken (lw), -O1 register allocation sayesinde tüm değişkenleri yazmaçlarda (a1, a2) tutar; bellek okuması 0'a iner.
Store sayısı	4	0	-O0 fonksiyon girişinde s0 ve gelen 3 argümanı stack belleğine yazar (sw); -O1'de argümanlar doğrudan a1/a2 yazmaçlarında kaldığı için stack'e hiç yazılmaz (register allocation). Fonksiyon yaprak (leaf) fonksiyon olduğundan ve spill gerekmediğinden stack frame açılmaz.

Ölçü	-O0	-O1	Farkın nedeni
Kullanılan register sayısı	8	3	-O0 stack yönetimi için sp, s0 ve argüman/yerel işlemler için a0, a1, a2, a4, a5, ra olmak üzere 8 register kullanır; -O1 leaf fonksiyon olduğundan yalnızca a1, a2 ve dönüş için ra (3 register) ile çalışır.

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
1	0x00400148	lui a5, 0x12f	U	imm[31:12]=00000000001001011111, rd=01111, opcode=0110111	0x0012F7B7	b7 f7 12 00	Evet
2	0x00400150	addi a5, a5, -1061	I	imm[11:0]=101111011011, rs1=01111, funct3=000, rd=01111, opcode=0010011	0xBDB78793	93 87 b7 bd	Evet
3	0x004000f0	lw a5, 0(a5)	I	imm[11:0]=00000000000, rs1=01111, funct3=010, rd=01111, opcode=0000011	0x0007A783	83 a7 07 00	Evet
4	0x00400100	sw a4, 0(a5)	S	imm[11:5]=0000000, rs2=01110, rs1=01111, funct3=010,	0x00E7A023	23 a0 e7 00	Evet

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
				imm[4:0]=00000, opcode=0100011			
5	0x004000e4	bge a5, a4, 68	B	imm[12|10:5]=0000010, rs2=01110, rs1=01111, funct3=101, imm[4:1|11]=00100, opcode=1100011	0x04E7D263	63 d2 e7 04	Evet
6	0x00400224	jalr ra, x6, -544	I	imm[11:0]=110111100000, rs1=00110, funct3=000, rd=00001, opcode=1100111	0xDE0300E7	e7 00 03 de	Evet
7	0x0040002c	jal zero, 136	J	imm[20|10:1|11|19:12]=00001000100000000000, rd=00000, opcode=1101111	0x0880006F	6f 00 80 08	Evet
8	0x00400134	jalr zero, ra, 0	I	imm[11:0]=000000000000, rs1=00001, funct3=000, rd=00000, opcode=1100111	0x00008067	67 80 00 00	Evet
9	0x00400044	add a5, a4, a5	R	funct7=0000000, rs2=01111,	0x00F707B3	b3 07 f7 00	Evet

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
				rs1=01110, funct3=000, rd=01111, opcode=0110011			
10	0x00400170	mul a4, a4, a5	R	funct7=0000001, rs2=01111, rs1=01110, funct3=000, rd=01110, opcode=0110011	0x02F70733	33 07 f7 02	Evet
11	0x00400a8	srai a5, a5, 2	I	imm[11:0]=010000000010, rs1=01111, funct3=101, rd=01111, opcode=0010011	0x4027D793	93 d7 27 40	Evet

Zorunlu instruction'ların tablodaki sıra numarası:

- ☑ SID'i yükleyen instruction(lar): # 1, # 2
- ☑ Koşullu branch: # 5
- ☑ Load: # 3
- ☑ Store: # 4
- ☑ Fonksiyon çağrısı / dönüşü: # 6 (call mov_avg), # 8 (ret)

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

> *Öğrenci numarası SID = 1240027 (0x0012EBDB), 32-bitlik pozitif bir tamsayıdır. RISC-V RV32I mimarisinde addi gibi standart immediate komutları 12-bit işaretli tam sayı kabul eder ve [-2048, +2047] aralığını aşamaz. 1240027 değeri bu sınırı aştığı için sabit tek bir komuta sığmaz; lui (üst 20 bit) ve addi (alt 12 bit) komut çiftiyle oluşturulur. 0x0012EBDB sayısının alt 12 biti 0xBDB (1011 1101 1011_2), üst 20 biti ise 0x0012E'dir. Alt 12 bitin en yüksek biti (bit 11) '1' olduğu için, donanım addi komutunu yürütürken işaret genişletmesiyle (sign extension) sayıyı 0xFFFFFbdb (-1061_10) negatif değerine dönüştürür. Bu negatif eksilmeyi telafi etmek için üst 20 bitlik lui değerine +1 düzeltmesi eklenir: 0x0012E + 1 = 0x0012F.*

> *Yürütme adımları:*
> *1. lui a5, 0x12F -> a5 = 0x0012F000 (1241088_10)*
> *2. addi a5, a5, -1061 -> a5 = 1241088 - 1061 = 1240027 (0x0012EBDB) tam değeri hatasız üretilir.*

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

> *Koşullu branch komutunun adresi PC = 0x004000e4 (bge a5, a4, .L9), hedef etiketin (.L9) adresi Target = 0x00400128'dir.*
> *1. Adres Farkı (Offset): Offset = Target - PC = 0x00400128 - 0x004000e4 = +0x44 = +68 byte.*
> *2. 13-bit İkiğe Tümleyen Binary: +68_10 = 0000001000100_2. En düşük bit (bit 0) çift byte hizalama gereği komutta saklanmaz; bit 12..1 komut içinde kodlanır.*
> *3. B-Format Scrambling (Bit Dağılımı):*
> - *imm[12] = 0*
> - *imm[10:5] = 000010*
> - *imm[4:1] = 0010*
> - *imm[11] = 0*
>
> *4. Komut Alanlarının Birleşimi: imm[12|10:5]=0000010, rs2=01110 (a4), rs1=01111 (a5), funct3=101 (BGE), imm[4:1|11]=00100, opcode=1100011 (BRANCH)*
> *5. Makine Kodu (Hex): 0000010 01110 01111 101 00100 1100011_2 = 0x04E7D263. Bu sonuç RARS dökümündeki 0x04e7d263 ile birebir örtüşür.*
> *6. Fonksiyon Çağrısı Offset Hesabı (#6): call mov_avg (auipc x6, 0 + jalr x1, x6, -544) komutunda hedef 0x00400000, call adresi 0x00400220'dir. Offset = 0x00400000 - 0x00400220 = -0x220 = -544 byte. jalr içindeki 12-bit signed immediate = 0xDE0 (-544) olup dönüş adresi ra (x1) yazmacına kaydedilir.*

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☐ Pipeline yok, her an tek instruction yürütülür.
- ☐ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☐ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☐ Data bus genişliği = ____ bit.

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

>

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										
14										
15										
16										
17										
18										
19										
20										

Kernel toplamları	Sayı
Instruction fetch	
Operand fetch (of)	
Operand store (os)	
PC'nin sıra dışı değiştiği instruction	

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1					
2					
3					
4					
5					
6					
7					
8					

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı		

Tek accumulator'ın getirdiği sınırlama (PDP-11 ADD B,A ile karşılaştırarak):

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR			
MAR			
IR			
IBR			
PC			
AC			
MQ			

3c. Interrupt senaryosu (k = ____, tür = ____):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	

Soru	Cevap
Kaydedilen PC değeri (hex)	
Kaydedilen diğer durum bilgisi	
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	
Kaynak (ISA referansı, bölüm, sayfa)	
Handler dönüşünde ilk çalışan instruction'ın adresi	

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?		
Dönüş adresini kim kaydeder, nereye?		
User program bundan haberdar mı?		

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir		
t_1: 2. interrupt gelir		
t_2		
t_3		
t_4: user program'a dönüş		

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)		
Data read		
Data write		
Toplam		

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek		
Data bus 16 bit olsaydı kernel'in bus cycle sayısı		

Hesap	Değer	Adımlar
Mevcut / 16-bit oranı		

PCIe paket verimi (N × 4 = ____ byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)		
Sequence number		
Header		
Data		
ECRC		
LCRC		
STP framing (son)		
Toplam		
Verim = data / toplam		
128b/130b ile verim		

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda `trace.csv` dosyasındadır. Biçim: `sira,RW,adres_hex,boyut,c_satiri,stack_mi`.

Ölçü	Değer
Kullanılan derleme (-O0 / -O1)	
Toplam data referansı	
Load / store	
Stack erişimi sayısı	
Ana diziye erişim sayısı	

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
22								
23								
24								
25								
26								
27								
28								
29								
30								

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0					
1					
2					
3					
4					
5					
6					
7					

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	
Hit			
Miss			
Hit ratio H			
Ana belleğe yazma sayısı			

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

- ☐ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = _____ B)	Hit	Miss	H
Direct			
2-way set-associative			
4-way set-associative			
Fully associative (LRU)			

Line boyu (mapping sabit = _____)	Hit	Miss	H
8 B			
16 B			
32 B			
64 B			

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 =	1
Kişiye özel cache			

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı		
Y7 çıktısı (aynı olmalı)		
H		
T_s (cycle)		

Değişikliğin gerekçesi; H artmadıysa nedeni:

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır):

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1		
Çarpma	3		
Load	1		
Store	1		
Branch / jump	2		
Ic (toplam)			
Ortalama CPI			

5a-2. Süre ve MIPS rate (f = ____ GHz, τ = ____ ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	$Ic \times CPI \times \tau$		
MIPS rate	$f / (CPI \times 10^6)$		
m (instruction başına data referansı)	data ref. / Ic		
T, cache yok (k = T2)	$Ic \times [p + (m \times k)] \times \tau$		
T, cache var (k = T_s)	$Ic \times [p + (m \times k)] \times \tau$		

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	
Speedup cache yok → cache var (-O0)	

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

5a-3. Amdahl yasası (Nc = ____):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	
Bağımsızlık gerekçesi (veri bağımlılığı)	
f (yürütme süresindeki pay)	
Speedup = $1 / [(1 - f) + f / Nc]$	
$Nc \to \infty$ limiti	

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing		
Data storage		
Data movement		
Control		

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☐ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☐ C kaynak kodu (`main.c`) ve sonraki 4g sürümü (`main_v2.c`)
- ☐ `asm_O0.s`, `asm_O1.s` ve `objdump -d` çıktısı
- ☐ `trace.csv` ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☐ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu
- ☐ 4e grafiği
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Opus 5.5	1.3-1.5	Üretilen assembly kodunun simülasyon ortamında yürütülme metodolojisi ve bellek segmenti inceleme adımları hakkında rehberlik almak için.	Bu assembly kodunu hangi emülatörde, nasıl ve nelere dikkat ederek inceleyebilirim? Bellekten çıktı değerlerini okurken nelere bakmalıyım?	Önerilen RARS simülatörü üzerinde kod koşturuldu. Data Segment alanındaki SID, alarm_count, first_alarm_idx ve arr değişkenlerinin bellek adresleri ile tutulan değerler adım adım kontrol edilerek doğrulandı.
Claude 3.7 Sonnet / Antigravity	0, 1 ve 2	Raporun kontrolü, parametre ve adres tutarlılığı, 2a C-assembly eşlemesinde döngü ve prologue tespiti, 2b optimizasyon analizi ve 2c branch/call makine kodu kodlamasında hata tespiti ve düzeltme önerileri almak için.	2a eşleme tablosunda döngü başlangıçları ve prologue'lar ile 2b tablosundaki register sayısı ve optimizasyon gerekçelerini kontrol edip doğrular mısın?	Hex değerleri, bit alanları ve adresler RARS v1.6 simülatörünün Text Segment Window dökümüyle ve resmi RISC-V Unprivileged ISA referansıyla karşılaştırılarak elle doğrulandı.

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		
Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (-O1 listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza